Reviewer Pack — Walsh-Code Antichain Width of NW Designs Bounds PRG Linear B…

Entry 52d3382c4243 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 14:54:46 UTC

Walsh-Code Antichain Width of NW Designs Bounds PRG Linear Bias

Entry ID: `52d3382c4243` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-28 14:54:46 UTC

1. Conjecture

Field A (mathematical branch): Extremal set theory: LYM/Dilworth antichain width of the design's intersection poset, scored by Walsh-character weight (a Sperner-LYM invariant rarely used in PRG analysis, distinct from Möbius/lattice or expander arguments) **Field B** (complexity object): Output linear (parity) bias of the Nisan-Wigderson generator NW_{D,f}: {0,1}^d -> {0,1}^m against parity distinguishers chi_T, T subset of [m], built from a hard function f:{0,1}^l -> {0,1}

Statement:

> Let D=(S_1,...,S_m) be an (l,a)-NW design over [d] with a<=log2(m), and let P_D be the poset on nonempty subsets of [m] ordered by T<=T' iff the symmetric-difference indicator of (union of S_i for i in T) is a 2-adic refinement of that of T'. Define the Walsh antichain width W(D)=max over antichains A in P_D of sum_{T in A} 2^{-|union_{i in T} S_i|}. We conjecture that for every Boolean f with correlation eps with parities of degree <=a, the maximum linear bias of NW_{D,f} satisfies bias(NW_{D,f}) <= W(D) * eps, and moreover W(D) = Theta(m * 2^{-a}) on standard Galois-field NW designs, giving a tight Sperner-style replacement for the union-bound m * 2^{-a} factor in Nisan-Wigderson.

Rationale (proposer's reasoning):

> The classical NW analysis pays a union-bound factor of m for the worst parity distinguisher, but the actual bias is a sum of Walsh coefficients indexed by subsets T of [m] whose contributions cancel along chains in the refinement poset; bounding cancellation by an LYM-type antichain width replaces the loose m factor with a combinatorially tight quantity. If the conjecture holds, NW seed length can shrink whenever W(D) « m, exposing a Sperner obstruction to natural-proofs-style attacks and giving a quantitative explanation for empirically observed sub-union-bound behavior.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3b9035fd14022b58`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across >=300 (D,f) pairs from Mod-p NW designs (d<=20, l in {3,4,5}, a in {2,3}, m in {4..16}), conjecture is SUPPORTED iff bias <= 1.05$W(D)$eps in >=99% of pairs AND mean(log2 W(D) - (log2 m - a)) lies in [-0.5,0.5]. Any single pair with bias > 1.05$W(D)$eps falsifies.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Nisan-Wigderson generator linear bias Walsh parity correlation design - LYM Sperner antichain pseudorandom generator Nisan-Wigderson combinatorial design - Walsh character weight intersection poset NW design pseudorandom bias`

Top relevant hits considered: - [http://arxiv.org/abs/1504.04131v2] Formulas for the Walsh coefficients of smooth functions and their application to bounds on the Walsh coefficients - [http://arxiv.org/abs/1905.12811v1] Embedding of Walsh Brownian Motion - [http://arxiv.org/abs/2601.03730v1 Perception-Aware Bias Detection for Query Suggestions - [http://arxiv.org/abs/1311.6531v3] Brains and pseudorandom generators - [http://arxiv.org/abs/1309.4930v2] The Zero-Undetected-Error Capacity Approaches the Sperner Capacity - [http://arxiv.org/abs/1712.00913v3] Majorization and Rényi Entropy Inequalities via Sperner Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import json
from itertools import combinations

# Helper functions for linear algebra
def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[sum(A[i][k] * B[k][j] for k in range(p)) for j in range(n)] for i in range(m)]
    return C

def gaussian_elimination(A, b):
    m = len(A)
```

```
    n = len(b)
    augmented = [A[i] + [b[i]] for i in range(m)]

    for i in range(m):
        # Find the pivot
        max_row = i
        for j in range(i+1, m):
            if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                max_row = j

        # Swap rows
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]

        # Eliminate non-pivot elements
        for j in range(i+1, m):
            factor = augmented[j][i] / augmented[i][i]
            for k in range(n + 1):
                augmented[j][k] -= factor * augmented[i][k]

    # Back-substitute to find the solution
    x = [0] * n
    for i in range(m-1, -1, -1):
        x[i] = augmented[i][n] / augmented[i][i]
        for j in range(i-1, -1, -1):
            augmented[j][n] -= augmented[j][i] * x[i]

    return x

# Helper functions for the conjecture
def walsh_coefficient(f, x):
    n = len(x)
    result = 0
    for i in range(2**n):
        sign = (-1) ** sum((x[j] & (1 << k)) != 0 for j in range(n))
        result += sign * f(i)
    return result / 2**n

def symmetric_difference_dimension(A, B):
    diff = A ^ B
    count = 0
    while diff:
        if diff & 1:
            count += 1
        diff >>= 1
    return count

def generate_NW_design(d, l, a, m):
    S = []
    for i in range(m):
        S.append(random.sample(range(2**d), random.randint(1, d)))
    return S

def compute_W(D):
    m = len(D)
    P_D = [[] for _ in range(1 << m)]
```

```
    # Generate the poset
    for T in range(1 << m):
        if T == 0:
            continue
        for i in range(m):
            if (T & (1 << i)) != 0:
                parent = T ^ (1 << i)
                P_D[parent].append(T)

    # Compute the antichain width
    W_D = 0
    for A in combinations(range(1 << m), len(A)):
        A_set = set(A)
        if all(symmetric_difference_dimension(T, U) % 2 == 1 for T in A_set for U in A_set if T !=
            W_D = max(W_D, sum(2**(-len(S[i] for i in T)) for T in A))

    return W_D

def run_trial(seed: int) -> dict:
    random.seed(seed)
    d = random.randint(1, 20)
    l = random.choice([3, 4, 5])
    a = random.choice([2, 3])
    m = random.randint(4, 16)

    S = generate_NW_design(d, l, a, m)
    P_D = [[] for _ in range(1 << m)]

    # Generate the poset
    for T in range(1 << m):
        if T == 0:
            continue
        for i in range(m):
            if (T & (1 << i)) != 0:
                parent = T ^ (1 << i)
                P_D[parent].append(T)

    # Compute the antichain width
    W_D = max(sum(2**(-len(S[i] for i in T)) for T in A) for A in combinations(range(1 << m), len(

    # Sample a hard function f as a random parity-balanced function
    x = [random.choice([0, 1]) for _ in range(d)]
    def f(i):
        return sum(x[j] & (1 << k) != 0 for j in range(d)) % 2

    eps = max(abs(walsh_coefficient(f, x)) for x in combinations(range(2**d), a))

    # Compute the maximum linear bias of NW_{D,f}
    max_bias = 0
    for T in range(1 << m):
        if T != 0:
            bias = abs(sum(f(i) for i in S[j] for j in range(m) if (T & (1 << j)) != 0))
            max_bias = max(max_bias, bias)

    # Check the conjecture
    conjecture_holds = max_bias <= W_D * eps
```

```
    return {
        "metric_name": "bias",
        "metric_value": max_bias,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"max_bias={max_bias} > W(D)*eps={W_D*eps}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_bias = sum(r["metric_value"] for r in results) / len(results)
    std_bias = math.sqrt(sum((r["metric_value"] - mean_bias)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_bias} std={std_bias} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"max_bias > W(D)*eps\" first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74c5ddfd.py", line 153, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_74c5ddfd.py", line 121, in run_trial
    W_D = max(sum(2**(-len(S[i] for i in T)) for T in A) for A in combinations(range(1 << m), len(A)))
                                                                                                  ^
NameError: name 'A' is not defined. Did you mean: 'a'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a NameError before producing any data, so the pre-registered support condition cannot be evaluated. No (D,f) pairs were tested. | next: Fix the buggy line computing W_D (the generator references 'A' inside its own definition and misuses combinations); compute W(D) by enumerating antichains in P_D properly, then rerun the 300-pair sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	22881
2	preregistration	claude_max	opus	0	0	8346
3	novelty	claude_max	opus	0	0	4768
4	novelty	claude_max	opus	0	0	9679
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11965
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17278
7	judge	claude_max	opus	0	0	5930

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 80846 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/52d3382c4243.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/52d3382c4243.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/52d3382c4243.tar.gz` (if generated)